```
>>>
         512 function calls in 64.020 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.004    0.004   64.020   64.020 <string>:1(<module>)
        1    0.034    0.034    0.037    0.037 matrix_mult.py:11(zero)
        1   63.980   63.980   64.016   64.016 matrix_mult.py:15(mult)
        7    0.000    0.000    0.000    0.000 {len}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
      501    0.003    0.000    0.003    0.000 {range}
```

Figure 4: After Runsnake-revealed optimisation

Runsnake by storing the output of cProfile in a file (e.g., *context.profile*) using the following command in the terminal:

```
python -m cProfile -o context.profile matrix_mult.py
```

Then visualise it using the command *runsnake context.profile*, which generates a window like what's shown in Figure 3. The Runsnake module is simple, and tells you how much time is spent in a given region of your code by scaling that region appropriately.

Optimisations

As you can see, we spend a lot of time in the *__init__* function as compared to the others, so let's try to get rid of the class matrix and replace it with just a simple function matrix and zero, which effectively does the same thing. The following modifications were made to the code:

```
#instead of a class matrix we define a function matrix and a
function for zero
def matrix(m, n):
      # Create random matrix
      mat = [[random.random() for row in range(n)] for col in
range(m)]
      return mat

def zero(m, n):
      mat = [[0 for row in range(n)] for col in range(m)]
      return mat
```

As you see in Figure 4, the execution time improvement is nearly 6 seconds, and the number of calls went from 251014 to 512! A simple optimisation like this can speed up code immensely; even though you can't tell from the cProfile output, you can get a clear picture using Runsnake, and act accordingly.

Line_profiler

Now let us look at the *line_profiler* library. cProfile may be fine and dandy for projects where you need a brief overview of which functions are slowing your code down, but what if you want to know which precise line in the function is responsible for the slowdown? Well, here's a neat tool, known as *line_profiler*. When you install the *line_profiler* library, you get a file called *kernprof.py*, which is responsible for profiling your code on a line-by-line basis, rather than as an entire function. To use *line_profiler*, first decorate with *@profile*:

```
@profile
def mult(matrix1, matrix2):
#rest of the code same as the unoptimised code
```

Note: After finishing profiling with *kernprof.py*, please remember to remove the *@profile*, because the Python interpreter will not understand the decorator.

Next, run *kernprof.py* with the *-l -v* flags to perform line-by-line profiling and to view the output, using the following command:

```
kernprof.py -l -v matrix_mult.py
```